Study B (contraception) — QC before analysis

Quality control: checking the data for problems **before** we run any tests. If a sample has almost no reads, or a table doesn't line up with the sample list, any test we run afterwards would be quietly wrong. Better to find that now.

There are three count-like tables in Study B: the microbiome (species), the RNA-seq (genes), and the cytokines. We check them one at a time.

```
# here() finds files relative to the project folder, no matter where the
# notebook is opened from. count_tables.R is the workshop helper that loads
# a dataset and lines up the tables for us.
library(tidyverse)
library(here)

source(here("R/count_tables.R"))

d <- read_counts("group_b")   # the contraception study
```

1. The sample table

The sample table should have exactly one row per sample, no duplicate IDs, and no missing values in the columns we plan to use.

```
# glimpse() prints the first few values of every column, one column per line
glimpse(d$metadata)
```

```
Rows: 93
Columns: 8
$ sample_id      <chr> "B001", "B002", "B003", "B004", "B005", "B006", "B007",~
$ participant_id <chr> "B001", "B002", "B003", "B004", "B005", "B006", "B007",~
$ arm            <chr> "pill", "injectable", "injectable", "ring", "pill", "ri~
```

```
$ bv             <chr> "negative", "negative", "negative", "positive", "positi~
$ nugent         <dbl> 1, 0, 0, 8, 8, 0, 0, 8, 0, 2, 10, 0, 0, 8, 0, 0, 8, 6, ~
$ ph             <dbl> 4.4, 5.3, 4.1, 5.3, 4.4, 5.0, 4.4, 5.3, 5.6, 5.3, 5.0, ~
$ any_sti        <dbl> 1, 1, 0, 0, 1, 1, 1, 0, 0, 1, 1, 1, 0, 1, 0, 1, 0, 0, 1~
$ rnaseq_batch   <chr> "B1", "B1", "B2", "B2", "B1", "B1", "B1", "B2", "B1", "~
```

```
# n() counts rows; n_distinct() counts how many DIFFERENT values there are.
# If every participant appears once, these two numbers are equal.
d$metadata |>
  summarise(samples            = n(),
            participants       = n_distinct(participant_id),
            duplicate_ids      = sum(duplicated(sample_id)),
            rows_with_any_NA   = sum(!complete.cases(pick(everything()))))
```

```
# A tibble: 1 x 4
  samples participants duplicate_ids rows_with_any_NA
    <int>        <int>         <int>            <int>
1      93           93             0                0
```

The three arms (contraceptive methods) and the two sequencing batches:

```
count(d$metadata, arm)
```

```
# A tibble: 3 x 2
  arm            n
  <chr>      <int>
1 injectable    39
2 pill          28
3 ring          26
```

```
count(d$metadata, rnaseq_batch)
```

```
# A tibble: 2 x 2
  rnaseq_batch     n
  <chr>        <int>
1 B1              47
2 B2              46
```

```
# Bacterial vaginosis (Nugent score) by arm - just so we know the groups
# aren't wildly lopsided in something related to our outcome.
d$metadata |>
  count(arm, bv) |>
  pivot_wider(names_from = bv, values_from = n)
```

```
# A tibble: 3 x 4
  arm        negative positive intermediate
  <chr>         <int>    <int>        <int>
1 injectable       20       19           NA
2 pill             16       10            2
3 ring             15       10            1
```

- **Check me:** the first table should show 93 samples and 93 participants, 0 duplicate IDs and 0 rows with missing values. You can also open `data/2-workshop2/group_b_contraception/01_metadata.csv` and see for yourself that it has 93 data rows.

2. Do the count tables line up with the sample table?

The count tables have samples as **columns**. Every test this week assumes the columns are in the same order as the rows of the sample table. If they matched by name but not by order, R would happily compare sample 5's counts with sample 6's metadata — and never warn us.

```
# identical() asks: are these two character vectors EXACTLY the same,
# in the same order? TRUE is what we want.
identical(colnames(d$species), d$metadata$sample_id)   # species table
```

```
[1] TRUE
```

```
identical(colnames(d$genes),   d$metadata$sample_id)   # gene table
```

```
[1] TRUE
```

Both come out `TRUE` because `read_counts()` puts the columns in metadata order as it loads them. Had we loaded the files by hand, this is the check we would have had to do ourselves.

3. Species table — depth and detection

Two basic questions per sample:

- **Library size** (also called *depth*): total reads in the sample. A sample with far fewer reads than its neighbours saw less, so its counts are noisier.
- **Features detected**: how many species were seen at least once. A sample that detected half as many species as usual may have been under-sequenced.

```
# Everything is computed column-wise: one column of the count matrix = one sample.
species_qc <- tibble(
  sample            = colnames(d$species),
  library_size      = colSums(d$species),
  features_detected = colSums(d$species > 0),
  # colMeans(d$species > 0) gives, per sample, the share of species DETECTED;
  # 1 minus that is the share of cells that are zero.
  fraction_zeros    = 1 - colMeans(d$species > 0)
)

species_qc
```

```
# A tibble: 93 x 4
   sample library_size features_detected fraction_zeros
   <chr>         <dbl>             <dbl>          <dbl>
 1 B001          54122                25          0.886
 2 B002          23843                43          0.805
 3 B003          14057                28          0.873
 4 B004          44211                37          0.832
 5 B005          27343                20          0.909
 6 B006           5247                24          0.891
 7 B007          31951                11          0.95
 8 B008          20556                48          0.782
 9 B009          18234                39          0.823
10 B010          27419                11          0.95
# i 83 more rows
```

```
# Two panels side by side: library size on the left, species detected on the
# right. geom_col() = a bar chart. coord_flip() turns the bars horizontal so
# the sample IDs stay readable.
species_qc |>
  mutate(sample = fct_reorder(sample, library_size)) |>   # sort bars by size
  pivot_longer(c(library_size, features_detected),
```

```
              names_to = "measure", values_to = "value") |>
  ggplot(aes(sample, value)) +
  geom_col(aes(fill = measure), show.legend = FALSE) +
  facet_wrap(~measure, scales = "free") +
  coord_flip() +
  labs(x = NULL, y = NULL,
       title = "Species table: every sample, sorted by library size")
```



The bars should step down smoothly. What we're looking for is a sample that drops far below the rest — a gap in the staircase, not the lowest bar.

```
# How bad is the worst sample, relative to a typical one?
species_qc |>
  summarise(median_depth = median(library_size),
            min_depth    = min(library_size),
            ratio        = min(library_size) / median(library_size))
```

```
# A tibble: 1 x 3
  median_depth min_depth ratio
         <dbl>     <dbl> <dbl>
1        29212      5247 0.180
```

```r
# Flag anything below half the median - a common, rough rule of thumb.
species_qc |>
  filter(library_size < median(library_size) / 2)
```

```
# A tibble: 9 x 4
  sample library_size features_detected fraction_zeros
  <chr>         <dbl>             <dbl>          <dbl>
1 B003          14057                28          0.873
2 B006           5247                24          0.891
3 B024          11627                39          0.823
4 B065          11570                28          0.873
5 B069          12927                13          0.941
6 B070          13333                42          0.809
7 B076          14534                18          0.918
8 B083          10756                20          0.909
9 B091          13543                 8          0.964
```

- **Check me:** `sum(d$species[, "B001"])` in the console should equal the `library_size` shown for B001 in the table above. (The square brackets are needed because `d$species` is a plain matrix — `d$species$B001` errors.)
- **Check me:** `mean(as.matrix(d$species) == 0)` should be about 0.88 — around 88% of cells in this table are zero. That is normal for microbiome data and it's why we treat it as *compositional*: a species' share went down is not the same claim as there was less of it.

4. Gene table — depth and detection

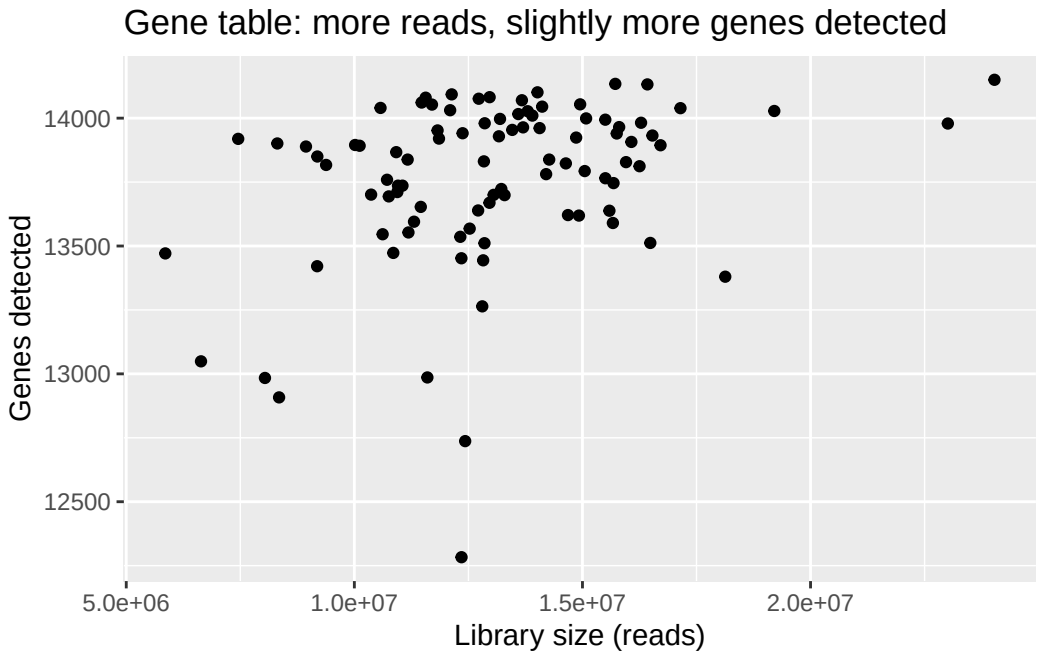
Same checks for the RNA-seq table. Depths here are in the millions, and detection is nearly total (most genes get at least a few reads in every sample), so the numbers look completely different — that's expected, not a bug.

```r
genes_qc <- tibble(
  sample            = colnames(d$genes),
  library_size      = colSums(d$genes),
  features_detected = colSums(d$genes > 0)
)

genes_qc |>
  summarise(across(c(library_size, features_detected),
                   list(min = min, median = median, max = max)))
```

```
# A tibble: 1 x 6
  library_size_min library_size_median library_size_max features_detected_min
             <dbl>               <dbl>            <dbl>                  <dbl>
1          5855848            12857804         24031761                  12283
# i 2 more variables: features_detected_median <dbl>,
#   features_detected_max <dbl>
```

```
genes_qc |>
  mutate(sample = fct_reorder(sample, library_size)) |>
  ggplot(aes(library_size, features_detected)) +
  geom_point() +
  labs(title = "Gene table: more reads, slightly more genes detected",
       x = "Library size (reads)", y = "Genes detected")
```

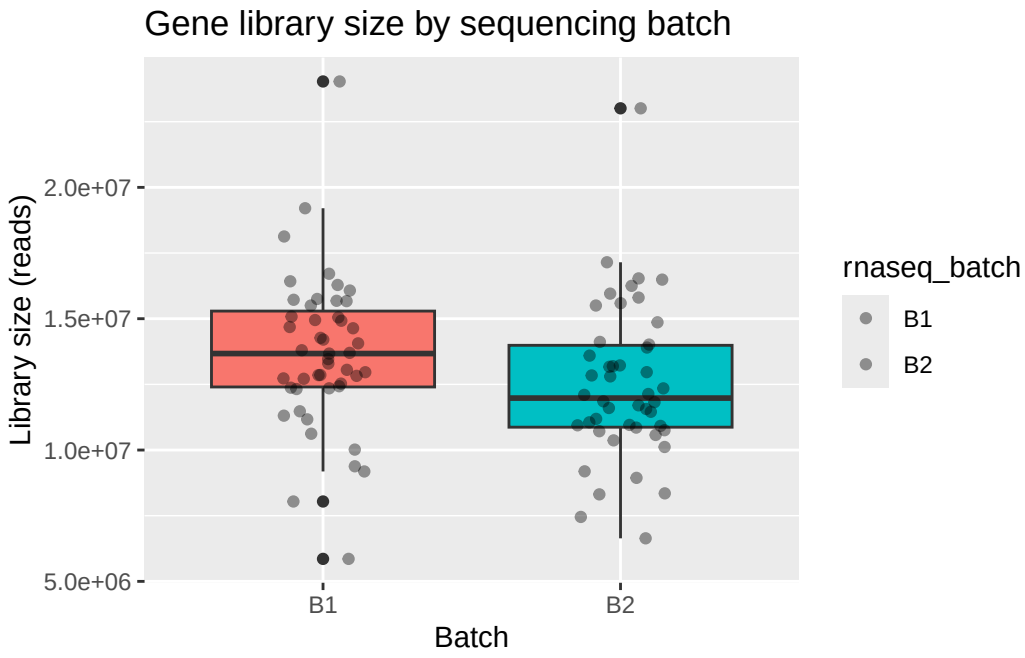


We want a point cloud drifting gently upward, with no point sitting alone in a corner. A sample far left of the rest is under-sequenced.

5. Sequencing batch

The gene table was sequenced in two batches (B1, B2). Batches are a classic source of technical differences that have nothing to do with biology, so we check whether batch and depth are tangled up with each other.

```r
genes_qc |>
  left_join(d$metadata, by = c("sample" = "sample_id")) |>   # bring the batch label over
  ggplot(aes(rnaseq_batch, library_size, fill = rnaseq_batch)) +
  geom_boxplot(show.legend = FALSE) +
  geom_jitter(width = 0.15, alpha = 0.4) +
  labs(title = "Gene library size by sequencing batch",
       x = "Batch", y = "Library size (reads)")
```




```r
# Are the arms spread across both batches, or is one arm all in one batch?
count(d$metadata, arm, rnaseq_batch) |>
  pivot_wider(names_from = rnaseq_batch, values_from = n)
```

```
# A tibble: 3 x 3
  arm           B1    B2
  <chr>      <int> <int>
1 injectable    20    19
2 pill          14    14
3 ring          13    13
```

If batch and arm were tangled, a real difference between arms would be impossible to tell apart from a batch effect. Here they're mixed, which is what we want — but keep batch in mind for the analysis.

- **Check me:** `count(d$metadata, arm, rnaseq_batch)` should show every arm appearing in both B1 and B2.

6. Cytokines

The cytokine table is measurements, not counts, so no depth check — but it can have missing values, which many tests refuse to handle.

```
# colSums(is.na()) counts missing values per cytokine; keep only the ones > 0.
tibble(cytokine = names(d$cytokines), missing = colSums(is.na(d$cytokines))) |>
  filter(missing > 0)
```

```
# A tibble: 15 x 2
   cytokine missing
   <chr>      <dbl>
 1 il1b           2
 2 il6            2
 3 tnfa           2
 4 ifny           2
 5 il10           2
 6 il4            2
 7 il17a          2
 8 il17f          2
 9 il21           2
10 il22           2
11 il23           2
12 il25           2
13 il31           2
14 il33           2
15 scd40l         2
```

- **Check me:** `sum(is.na(d$cytokines))` in the console should match the sum of the `missing` column above (30 at time of writing).

Verdict

- **Sample table**: clean — 93 samples, one per participant, no missing values, arms reasonably balanced.
- **Alignment**: both count tables line up with the metadata, in order.

- **Species table**: ~88% zeros (normal), depths span roughly 5k to 77k reads. The low end is thin; check the flagged samples above before differential testing.
- **Gene table**: millions of reads per sample, nearly all genes detected, depths within a few-fold of each other. Batches are balanced across arms.
- **Cytokines**: a handful of missing values to handle when we get there.

No sample needs to be dropped on the evidence so far. The analysis notebooks should use **raw counts** (not CPM, not logged) for DESeq2, and should keep an eye on batch.

Choices made

- Flagged low-depth samples at **below half the median** library size; an alternative is the bottom decile, or a fixed threshold, which would flag a different set.
- Checked **library size and features detected** as the two core QC metrics; an alternative is adding Shannon diversity here, which notebook 03 covers anyway.
- Plotted QC **per sample** rather than summarising to per-arm means, because QC problems are sample-level, not group-level.
- Checked batch balance **descriptively** (a table and a boxplot); an alternative is testing it formally, which we save for the analysis itself.